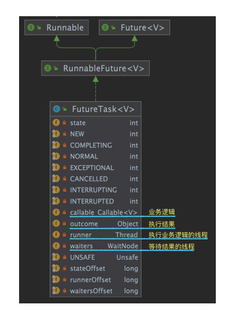

```
Callable, Future, FutureTask and CompletableFuture
在Java早期，创建线程有2种方式，继承Thread类or实现Runnable接口，这2种方式有一个共同的缺陷：run()没有返回值，也不向调用方抛异常
一个线程跑完就完了，调用方无法得知结果，只能靠共享变量、回调、或者日志来间接感知
因此，Java5引入了3个相辅相成的核心抽象 > Callable：「有返回值」「会抛异常」的Runnable
                                    > Future：「异步计算」的结果句柄，可查进度，获取结果，取消任务
                                    > FutureTask：Callable的「任务」+ Future的「结果」
                                    > CompletableFuture：Future的增强版，增加了任务编排，回调，和组合能力，适用于需要链式处理和结果聚合的异步计算场景
                                                        支持注册多个回调函数，任务完成自动触发下一步操作，像流水线一样把多个异步操作串连起来
有了这套机制，异步编程变得可控：将任务交给线程池执行，保留一个Future句柄，随时查看进度，设置超时，取消执行，捕获异常，读取结果

线程Thread和任务Task的关系？线程池的出现？
Thread代表操作系统层面的一个执行线程，Runnable和Callable代表「要执行的任务」
Thread是相对昂贵的资源，创建和销毁的开销比较大，为了将「Thread和Runnable/Callable解耦」，引入了线程池
Executors有完善的Thread管理机制，工作线程可复用，减少开销，控制最大并发线程数，使得「线程管理」和「业务逻辑」解耦

Callable的典型使用场景  > 并行计算：一个大任务，拆成多个小任务，封装成Callable，并行执行，最后合并结果
                      > 远程服务调用：封装HTTP、RPC、数据库等远程调用，拿到返回值后继续执行
                      > 文件读写：异步读取多个文件/处理多个数据块，返回处理结果
```

```java
FutureTask(Callable<V> callable) {
  if (callable == null) throw new NullPointerException();
  this.callable = callable;
  this.state = NEW;
}
```

```java
FutureTask(Runnable runnable, V result) {
  this.callable = Executors.callable(runnable, result);
  this.state = NEW;
}
```

```java
<T> Callable<T> callable(Runnable task, T result) {
  if (task == null) throw new NullPointerException();
  return new RunnableAdapter<T>(task, result);
}
```

```java
static final class RunnableAdapter<T> implements Callable<T> {
    final Runnable task;
    final T result;
    RunnableAdapter(Runnable task, T result) {
        this.task = task;
        this.result = result;
    }
    public T call() {
        task.run();
        return result;
    }
}
```

```java
Thread.start() -> FutureTask.run() { //新建线程执行task
  //1.如果state不是NEW，任务已执行过，直接返回；并将当前线程写入runner，保证一个FutureTask只被一个线程执行一次
  if (state != NEW || !UNSAFE.compareAndSwapObject(this, runnerOffset, null, Thread.currentThread())) return;
  try {
    Callable<V> c = callable; //要执行的异步任务
    if (c != null && state == NEW) {
      V result;
      boolean ran;
      try {
        result = c.call(); //2.执行任务，并保存结果
        ran = true; //正常结束标志
      } catch (Throwable ex) {
        result = null;
        ran = false;
        setException(ex); //3.抛异常了，保存异常信息，唤醒等待结果的线程
      }
      if (ran) set(result); //4.保存结果，唤醒等待结果的线程
    }
  } finally { //5.执行结束，清理工作
    runner = null; //避免已经结束的任务仍然持有线程引用
    int s = state;
    if (s >= INTERRUPTING) handlePossibleCancellationInterrupt(s);
  }
}
```

```java
void setException(Throwable t) {
  if (UNSAFE.compareAndSwapInt(this, stateOffset, NEW, COMPLETING)) { //1.CAS终止标志
    outcome = t; //保存异常信息
    UNSAFE.putOrderedInt(this, stateOffset, EXCEPTIONAL); // cas final state
    finishCompletion();
  }
}
```

```java
void set(V v) {
  if (UNSAFE.compareAndSwapInt(this, stateOffset, NEW, COMPLETING)) { //1.CAS终止标志
    outcome = v; //保存结果
    UNSAFE.putOrderedInt(this, stateOffset, NORMAL); // cas final state
    finishCompletion();
  }
}
```

```java
void finishCompletion() {
  //唤醒所有等待结果的线程
  for (WaitNode q; (q = waiters) != null;) { //waiters为空时，结束循环
    if (UNSAFE.compareAndSwapObject(this, waitersOffset, q, null)) { //将当前节点移出等待队列
      for (;;) {
        Thread t = q.thread;
        if (t != null) { //唤醒等待线程
          q.thread = null;
          LockSupport.unpark(t);
        }
        WaitNode next = q.next; //处理下一个节点
        if (next == null) break;
        q.next = null;
        q = next;
      }
      break;
    }
  }
  done();
  callable = null;
}
```

```java
ExecutorService executorService = Executors.newFixedThreadPool(10);
Callable<V> callable = new Callable<V>() { /* 定义业务逻辑 */ }

//提交线程池执行task 方式1:
FutureTask<V> futureTask = new FutureTask<V>(callable); //封装成可直接run的FutureTask
executorService.execute(futureTask); //直接执行

//提交线程池执行task 方式2:
Future<v> futureTask = executorService.submit(callable); //提交线程池，由线程池封装，并保存返回的futureTask

//获取结果
V result = futureTask.get();
```

```java
<T> Future<?> AbstractExecutorService.submit(Callable<T> task) {
  if (task == null) throw new NullPointerException();
  RunnableFuture<T> ftask = newTaskFor(task); //包装FutureTask: new FutureTask<T>(task)
  execute(ftask); //交给工作线程执行task，同executorService.execute()
  return ftask; //返回包装的FutureTask，调用者可以查看结果
}
```

很多初学者会混淆 execute() 和 submit()。它们的主要区别如下：

维度	execute(Runnable)	submit(Callable/Runnable)
入参类型	只能接收 Runnable	可接收 Callable 或 Runnable
返回值	void，无返回值	返回 Future，可获取结果和异常
异常处理	任务异常只能靠自定义机制处理	异常可通过 Future.get() 捕获
底层关系	直接执行任务	内部包装为 FutureTask 后调用 execute()

可以用一句话概括：submit() 是 execute() 的增强版，它在 execute() 的基础上额外解决了「结果回传」和「异常传递」两个问题。如果不需要结果，统一使用 execute()；需要结果时则使用 submit()。

```java
V FutureTask.get() throws InterruptedException, ExecutionException { //获取任务结果
  int s = state;
  if (s <= COMPLETING) s = awaitDone(false, 0L); //任务还在执行，阻塞当前线程，等待结果
  return report(s); //返回结果 or 抛异常
}
```

```java
V report(int s) throws ExecutionException{
  Object x = outcome;
  if (s == NORMAL) return (V)x;
  if (s >= CANCELLED) throw new CancellationException();
  throw new ExecutionException((Throwable)x);
}
```

```java
int awaitDone(boolean timed, long nanos) throws InterruptedException{
  final long deadline = timed ? System.nanoTime() + nanos : 0L;
  WaitNode q = null;
  boolean queued = false;
  for (;;) { //自旋，直到拿到结果
    if (Thread.interrupted()) { //CASE1:线程被打断，取消等待，抛出异常
      removeWaiter(q);
      throw new InterruptedException();
    }
    int s = state;
    if (s > COMPLETING) { //CASE2:有结果了，取消等待，返回最终状态
      if (q != null) q.thread = null;
      return s;
    }
    else if (s == COMPLETING) Thread.yield(); //CASE3:结果正在保存中，放弃CPU
    //CASE4:任务还在执行
    else if (q == null) q = new WaitNode();
    else if (!queued) queued = UNSAFE.compareAndSwapObject(this, waitersOffset, q.next = waiters, q); //加入等待队伍（单向链表），结果出来了会通知所有等待的线程
    else if (timed) { LockSupport.parkNanos(this, nanos);}
    else LockSupport.park(this); //阻塞当前线程，等待结果
  }
}
```